C++14 / GRAPH THEORY / 学习详解

格子不相邻选择

从题意到二分图最大匹配
定理证明 · DFS 过程 · 源代码逐段精读

二维坐标版　完整可编译代码　配套图示

4 个可选格子

	1	2	3
1	1	1	1
2	0	1	0

最多选 3 个

	1	2	3
1	X	1	X
2	0	X	0

答案 = 值为 1 的格子数 − 二分图最大匹配数

不只记住一个公式。本手册从“什么叫相邻、什么叫一对”开始，逐步说明为什么可以黑白分组，为什么减去的是配对数，以及 DFS 怎样通过修改旧配对找到更多配对。

> **围绕你追问的四个问题展开：**
> 所有的 1 都参与了吗？matchRight 到底记什么？为什么递归找原来的黑格子？`old.row == 0 || dfs(...)` 在什么时候执行赋值？

题目范围：1 ⩽ n,m ⩽ 20；矩阵仅含 0 和 1；只禁止上下左右相邻。
配套源码：grid_independent_set.cpp（88 行）
本文图示与例题推演为重新绘制；定理来源列于第 30 页。

学习笔记 / 02

阅读路线与符号约定

建议先读懂格子与配对，再读代码；不要一开始就把“独立集、点覆盖、增广路”三个新概念混在一起。页码均指本 PDF 的实际页码。

页码	主题	读完需要回答的问题
3–6	题意、黑白分组、匹配	每一个 1 去哪里了？为什么最多只能配一对？
7–8	答案公式的来源	为什么不是减去 2 倍匹配数？
9–10	增广路与 Berge 定理	为什么调整旧配对可以变得更好？
11–13	Kőnig 定理的构造证明	为什么最少放弃数恰好等于匹配数？
14–16	算法结构与二维数据	matchRight、vis、dfs 分别负责什么？
17–20	递归全流程与关键语句	何时改配、何时返回、为什么不能清空配对？
21	整套算法正确性	为什么每个黑点启动一次 DFS 就够了？
22–25	完整代码与逐段分析	能否把每句代码翻译成一句话？
26–29	验证、恢复方案、易错点、自测	怎样检查程序，怎样迁移到其他题？
30	参考资料与复习卡片	哪些结论来自定理，哪些来自本题建模？

统一符号，避免一词多义

符号 / 变量	含义
V、E	可选格子构成的顶点集合、相邻关系构成的边集合。
L、R	黑色点集合、白色点集合；不是矩阵几何上的左半边和右半边。
M、|M|	一个匹配及它的边数；最大匹配的边数记为 ν(G)。
I、C	独立集（保留的点）、点覆盖（准备放弃的点）。
α(G)、τ(G)	最大独立集大小、最小点覆盖大小。
total、matching	代码中分别统计 1 的个数、成功建立的配对数。

> 零基础路线：3–6 → 14–20 → 22–25。理解代码后，再回读 7–13 的定理证明。

学习笔记 / 03

先读懂题目：只让选中的格子互不相邻

数字 1 表示“允许选择”，不是“已经选择”。数字 0 不能选择。用 X 标记最终选中的格子。两个格子只有共用一条边才算相邻；斜对角只共用一个角，不算相邻。

样例 1：原矩阵

	1	2	3
1	1	1	1
2	0	1	0

合法方案：选择 3 个

	1	2	3
1	X	1	X
2	0	X	0

摆法	是否合法	原因
一行里连续两个 X	不合法	左右有公共边。
一列里连续两个 X	不合法	上下有公共边。
X 在斜对角上	合法	只在角上接触，没有公共边。
同一行 X 1 X	合法	中间的 1 没有被选中；两端 X 不相邻。

为什么样例 1 的答案是 3?

给四个 1 分别命名：第一行 A、B、C，第二行中间为 D。B 同时邻接 A、C、D。选 B 会排除另外三个；不选 B，则 A、C、D 可以一起选。

选 3 个已经可行，而选 4 个意味着四个 1 全选，必然有冲突。因此最大值恰好为 3。这里是在证明“至少为 3”与“至多为 3”，两边结合才得到最优答案。

> **输出的是最大数量，不是方案数。** 即使有好几种最优摆法，也只输出它们共同的格子数量。本题不要求输出坐标。

学习笔记 / 04

样例 2：给出一个方案，还要证明不能更多

样例 2：12 个可选点

	1	2	3	4
1	1	1	1	1
2	1	0	0	1
3	1	0	0	1
4	1	1	1	1

可以同时选择 6 个

	1	2	3	4
1	X	1	X	1
2	1	0	0	X
3	X	0	0	1
4	1	X	1	X

上图每行分别选 2、1、1、2 个，共 6 个，而且没有两个 X 共用一条边。所以答案至少为 6。

怎样证明最多只能选择 6 个?

把 12 个可选格子分成 6 对，每对中的两个格子都相邻。下面相同字母代表同一对：

6 对互不重复的相邻格子

	1	2	3	4
1	A	A	B	B
2	C	0	0	D
3	C	0	0	D
4	E	E	F	F

每对最多保留一个，所以总数最多为 6。前面又找到了恰好选 6 个的方案，因此答案就是 6。

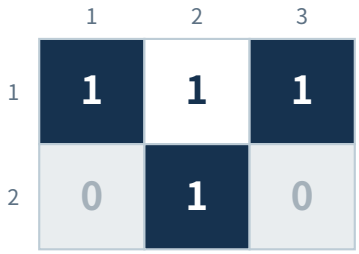
> 这已经暗含“匹配”的思想：找一些没有重复用点的相邻对，每一对都迫使我们至少放弃一个点。但一般题目还需要定理保证这个下界能够达到。

学习笔记 / 05

建图与黑白染色：所有的 1 都参与

把每个值为 1 的格子看成一个点；两个可选格子上下或左右相邻，就连一条边。边表示“这两个点不能同时被选中”的冲突关系。0 不建点，也不与其他点连边。

颜色由坐标决定；数字决定能否选择




```
黑色组：  (i + j) % 2 == 0
白色组：  (i + j) % 2 == 1
```

移动到上、下、左、右任意邻居，i+j 都恰好改变 1，奇偶性必然改变。因此每一条边都连接一黑一白。这满足二分图的定义：顶点可分成两组，所有边都跨组连接。[1]

两个容易混淆的判断

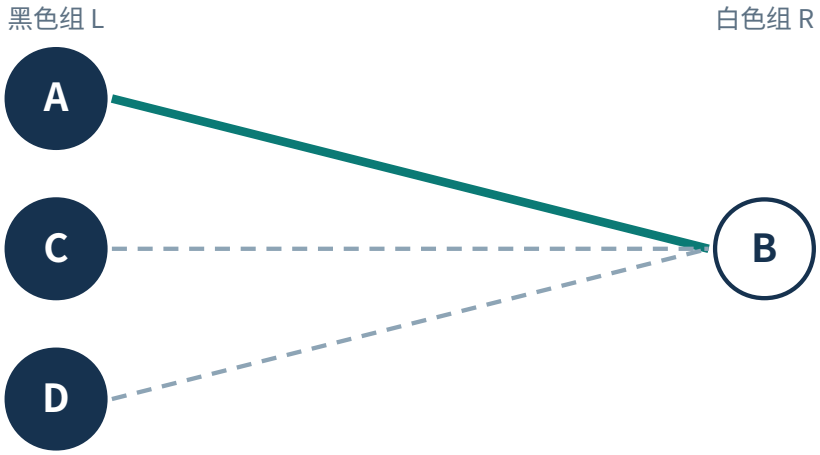
问题	正确理解
黑色是不是 0，白色是不是 1?	不是。颜色看坐标，0/1 看输入，二者相互独立。黑色 1、白色 1 都参与。
能否把任意黑点与任意白点配对?	不能。还必须有边，也就是原矩阵中上下左右相邻。
只从黑点启动 DFS，会不会漏掉白点?	不会。白点是寻找的目标，每条合法边都会在它的黑端被检查。
答案能否只选黑点或只选白点?	这两种选法都合法，但不一定最优；最优方案可以同时包含两种颜色。

> 反例：一行 `1 0 0 1` 中，两个 1 颜色不同却不相邻，可以同时选。只取“黑点数、白点数的较大者”只会得到 1，而正确答案是 2。

学习笔记 / 06

匹配到底在数什么：数“对”，不是数格子

匹配是一组边，其中每个点最多出现在一条被选中的边里。最大匹配就是在所有合法匹配中，边数最多的一组。它允许有点未配对，不要求完美覆盖所有点。[1][3]



样例 1 有三条边 A—B、C—B、D—B，但它们共享 B，不能同时作为匹配。最多只能选其中一条边，因此最大匹配数是 1，不是 3。

“随便配，直到配不了”还不够

```
一行四个可选点：A — B — C — D

先配 B–C：暂时只有 1 对，A、D 不能直接再配。
改成 A–B、C–D：可以得到 2 对。
```

第一种是“极大匹配”：保持已有配对不动时，不能直接再加一条边。第二种才是“最大匹配”：即使允许重新安排，也不存在更多对。不要把“极大”和“最大”混用。

概念	规则	本题中的作用
匹配	相邻点成对，不能重复使用点	用来计算必须放弃的最少数量。
独立集	任何两个选中的点都不相邻	这才是题目真正要找的格子集合。

> **配上的两个点不能同时作为答案选中。** “属于匹配”和“最终被选择”是两种完全不同的身份。

学习笔记 / 07

第一条关系：独立集与点覆盖互为补集

先不讨论算法。设 V 是全部可选点，我们最终保留一部分 I，并放弃另一部分 C，也就是 C = V − I。

名字	定义	格子语言
独立集 I	I 内部没有任何一条边	保留的格子之间没有相邻冲突。
点覆盖 C	每条边至少有一个端点属于 C	每一对相邻冲突，至少放弃其中一个格子。

方向一：I 合法，为什么 C 一定覆盖全部边？

任取一条边 u—v。如果两个端点都不在 C，它们就都在 I。这意味着两个被选中的点相邻，与 I 的独立性矛盾。因此至少一个端点在 C，所有边都被覆盖。

方向二：C 覆盖全部边，为什么剩余的 I 一定合法？

如果剩余的 I 中仍有两个点相邻，那条边的两个端点都不在 C。这条边没有被覆盖，矛盾。因此 I 内部没有边。

保留的点最多 ⇔ 放弃的点最少

$$\alpha(G) + \tau(G) = |V|$$

这里 α(G) 是最大独立集大小，τ(G) 是最小点覆盖大小。因为每个合法 I 都对应一个合法 C，反过来也一样，且它们的数量之和始终为 |V|，所以“最大”和“最小”也一一对应。

> 这一条补集关系对任意无向图都成立，不要求二分图。真正需要二分图条件的是下一步：用最大匹配大小替代最小点覆盖大小。

不要混淆“点覆盖”和“边覆盖”：这里挑选的是一些点，让这些点接触到所有边；不是挑边去覆盖所有点。

学习笔记 / 08

Kőnig 定理：最小点覆盖等于最大匹配

定理：对于二分图，最大匹配的边数，等于最小点覆盖的点数。它把“最少放弃多少个格子”转化成“最多能配多少对”。[1][3]

$$\tau(G) = \nu(G) \quad （前提：G 是二分图）$$

先证明容易的一半：任何点覆盖都不少于匹配数

假设有 k 条匹配边。这 k 条边的端点互不重复。点覆盖必须为每条匹配边至少选择一个端点，而且同一个点不能同时负责两条不同的匹配边。于是至少需要 k 个点。

$$\text{任何点覆盖大小} \geqslant \text{任何匹配大小}$$

取最小点覆盖和最大匹配，得到 $\tau(G) \geqslant \nu(G)$。这一方向对一般图也成立。难点在反方向：为什么二分图中一定能找到大小恰好为 $\nu(G)$ 的点覆盖？第 11–12 页会给出构造证明。

现在把两条关系连起来

```
最大独立集大小 = 全部点数 - 最小点覆盖大小
               = 全部点数 - 最大匹配数
```

因此代码输出 `total - matching`。减去的是“对数”，不是 2 倍对数；每一对迫使我们至少放弃一个，而不是两端都放弃。

> **不能把这个等式套到非二分图。** 三角形有 3 个点，最大匹配只有 1 对，但最多只能选 1 个互不相邻的点。3−1=2 是错误答案。这说明黑白染色不是装饰，而是在验证定理的适用条件。

反过来，“任意从每一对中删一个”也不构成证明；剩下的点之间可能仍有非匹配边。必须构造一个覆盖所有边的点集。

学习笔记 / 09

增广路：沿一条链调整，配对数增加 1

相对于当前匹配 M，一条路上的边如果交替为“非匹配边、匹配边、非匹配边……”，称为交替路。如果它的两个端点都未配对，就叫增广路。二分图中可以从一个未配对黑点出发，到达一个未配对白点。[2]



例子里已有 A—P，B 和 Q 都未配对。B—P—A—Q 是一条增广路：B 想找 P，P 已属于 A，A 又能找到空闲 Q。

翻转路径上的边

把路径上原来已匹配的边撤销，把原来未匹配的边加入。图中就从 A—P 变成 B—P、A—Q。



比较项目	翻转前	翻转后
路径上匹配边数量	1 条：A—P	2 条：B—P、A—Q
内部点 A、P	各属于 1 条匹配边	仍各属于 1 条匹配边
两个端点 B、Q	未配对	现在都配对
路径之外	保持原状	保持原状

一般地，若路径有 k 条匹配边，就有 k+1 条非匹配边。翻转后净增加一条匹配边，而且没有点参加两对。这就是为什么外层一次成功只执行一次 `matching++`。

> “帮原对象换一个位置”是直观说法；“沿增广路翻转匹配状态”是同一过程的图论描述。格子本身始终没有移动。

学习笔记 / 10

Berge 定理：没有增广路，才真正最大

定理：一个匹配是最大匹配，当且仅当相对于它不存在增广路。这个结论适用于一般无向图；本题在二分图中寻找增广路更容易。[2]

方向一：有增广路，就一定还不是最大

第 9 页已经说明，翻转一条增广路会得到多一条边的新匹配。如果还能增加，原来的匹配当然不是最大。

方向二：不是最大，就一定存在增广路

设当前匹配为 M，假设还有一个更大的匹配 M*。比较两者，把它们共有的边删去，只保留“只属于其中一个匹配”的边，这叫对称差。

每个点在 M 中最多连一条边，在 M* 中也最多连一条边。所以保留下来的图，每个点最多连两条边，只能分解成若干条路和若干个环，而且两种边交替出现。

分解得到的部分	两种边的数量关系
交替环	M 与 M* 的边一样多。
偶数条边的交替路	M 与 M* 的边一样多。
从 M 边开始、以 M 边结束的路	M 比 M* 多一条。
从 M* 边开始、以 M* 边结束的路	M* 比 M 多一条。

由于 M* 的总边数更多，至少有一条路属于最后一种。它的两端不可能还接有 M 中的匹配边，否则就不是该分量的端点；也不可能有被删去的公共匹配边，否则会违反 M* 中一个点只能用一次。

因此，这条路的两个端点在 M 中都未配对，路上的边又交替出现，它正是一条 M 的增广路。这与“没有增广路”的假设矛盾，证明完成。

> **这一定理保证的是最优性，不只是局部合理。** 贪心可能找不到可直接添加的一对，但通过一串改配仍然能增加；DFS 必须允许递归调整旧配对。

学习笔记 / 11

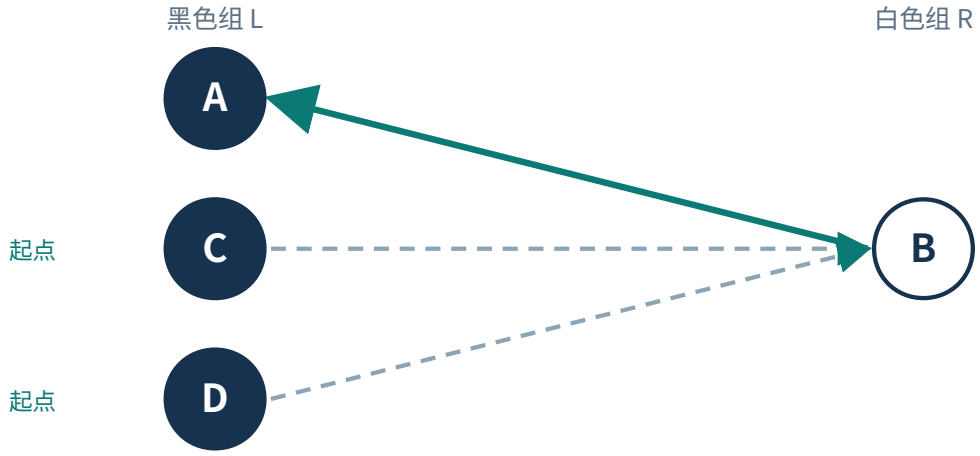
Kőnig 定理的构造：从未配对黑点出发

现在补全第 8 页中困难的一半。先取一个最大匹配 M，把全部点分成黑色组 L、白色组 R。下面构造一个恰有 |M| 个点的点覆盖。[1]

第一步：按匹配关系确定搜索方向

```
非匹配边：黑点 → 白点
匹配边：  白点 → 它已经配对的黑点
```

从所有未配对黑点同时开始搜索。把能到达的黑点记为 ZL，能到达的白点记为 ZR。起点本身属于 ZL。这里的搜索是证明和恢复方案使用的搜索，不是直接复用最后一轮 DFS 的 vis。



图示沿用样例 1，假设匹配 A—B。C、D 是未配对黑点，因此是搜索起点。沿 C→B→A、D→B，可以到达全部四个点。

第二步：按“黑不达、白可达”取点

$$C = (L - ZL) \cup ZR$$

中文读法：把**没有到达的黑点**和**已经到达的白点**放进 C。这个集合将成为最小点覆盖，也就是应该放弃的格子。

> 两个必须记住的事实：
> ① 可达白点一定已匹配，否则找到了一条增广路，与 M 最大矛盾。
> ② 对一条匹配边，两个端点要么都可达，要么都不可达。下一页解释第二点并完成证明。

学习笔记 / 12

构造为什么正确：既覆盖全部边，又恰好够小

事实：匹配边的两个端点具有相同的可达性

设 u—v 是匹配边，u 是黑点，v 是白点。若 v 可达，沿匹配边 v→u 就能到达 u。若 u 可达，由于 u 已匹配，它不是起始的未配对黑点；搜索到它的最后一步只能是其唯一匹配对象 v→u，所以 v 也可达。

证明一：C 覆盖了所有边

任取边 u—v，u 为黑点。如果 u 不可达，u 已经属于 C，边被覆盖。如果 u 可达，继续分情况：

这条边的身份	为什么白点 v 也可达
非匹配边	搜索允许沿 u→v 前进，因此 v 属于 ZR。
匹配边	由上面的事实，匹配边两个端点同时可达。

两种情况里 v 都属于 ZR，也就属于 C。每一条边都有一个端点落在 C，因此 C 确实是点覆盖。

证明二：C 的大小恰好等于 |M|

C 中没有未匹配点：不可达黑点不可能未匹配，因为所有未配对黑点都是搜索起点；可达白点也不可能未匹配，否则出现增广路。

一条匹配边的状态	进入 C 的端点
两端都可达	黑点不选入 C；白点选入 C。
两端都不可达	黑点选入 C；白点不选入 C。

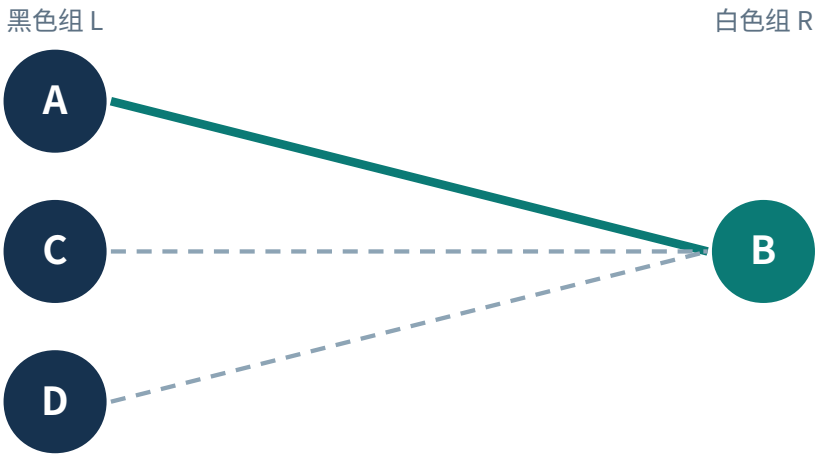
所以每条匹配边恰好贡献一个点，且 C 中每个点都属于某条匹配边。于是 $|C|=|M|$。结合之前 $\tau(G)\geqslant|M|$ 的下界，得到 $\tau(G)=|M|=\nu(G)$。

答案 = |V| − |M|

这份构造还给出了具体最优选择：I = ZL ∪ (R − ZR)，也就是保留“可达黑点”和“不可达白点”。数量最优，且没有任何相邻冲突。

学习笔记 / 13

把定理放回格子：为什么不能随便删一个



还是样例 1。匹配边只有 A—B；搜索起点是未配对黑点 C、D。可达集合是 ZL={A,C,D}、ZR={B}。

```
最小点覆盖 C = (L - ZL) ∪ ZR = {B}

最大独立集 I = V - C = {A, C, D}

答案 = 4 - 1 = 3
```

为什么匹配里删掉 A 不行?

假如只看配对 A—B，任意删掉 A，剩下 B、C、D。虽然匹配边 A—B 被消除了，但非匹配边 B—C、B—D 仍存在，所以剩余格子仍然相邻。

点覆盖要求覆盖的是**所有原图的边**，不只是匹配中的边。最大匹配告诉我们最少需要删几个，交替可达性进一步告诉我们怎样删才能真的做到。

> **回答两个常见疑问：**
> 为什么不减 2×matching？一对中至少放弃一个，不是两个都放弃。
> 未配对的点是不是都该直接选？不能孤立地做这个决定；选中集合应通过点覆盖的补集保证全局合法。

只输出数量时，不必真的恢复方案

本题只问最多能选几个。既然定理已经证明最小点覆盖的大小等于最大匹配大小，程序只需算出匹配数即可。恢复点覆盖是帮助理解、验证或处理“输出具体格子”变式的额外步骤。

学习笔记 / 14

算法总览与复杂度

阶段	做什么	得到什么
读入	读取每个 0/1，并累加所有 1	total
分组	按 (i+j) 的奇偶性把可选点分成黑白	每条边都跨黑白组
逐个尝试	每个黑色 1 启动一次 DFS；先清空本轮 vis	成功则配对数增加 1
递归改配	遇到已占用白点，就尝试帮助原黑点改配	沿增广路更新 matchRight
输出	根据两个定理，把匹配数换成最大独立集大小	total − matching

为什么不用建立邻接表?

每个格子最多只有 4 个邻居，坐标可以直接计算。DFS 里遍历四个方向，就相当于按需生成邻接表。二维实现因此可以省略 `id()` 和 `vector<int> g[]`。

范围有多大?

设 K=n×m 是格子数，V 是可选点数。V≤K≤400。所有上下左右相邻关系至多为 n(m−1)+(n−1)m；20×20 满矩阵时是 760 条无向边。

一次完整寻找中，白点至多访问一次；通过匹配边递归到黑点，再检查它的四个邻居。普通邻接表写法的增广搜索为 O(V+E)，处理全部黑点得到 O(V(V+E)) 的上界。[2]

本题：时间 $O((nm)^2)$，空间 $O(nm)$

固定 N=25 的实现每轮清零 625 个布尔值，在给定范围内很小。按输入规模动态分配时，标记、配对表、矩阵与递归栈都为 O(nm)。当前源码固定数组本身使用常量容量。

> DFS 的递归深度与可选点数相关，不会枚举 2^400 个选择。它在“找改配链”，不是尝试所有格子子集。

学习笔记 / 15

两个二维数组：一个记关系，一个记本轮足迹

```
struct Point {
    int row;
    int col;
};

Point matchRight[N][N];
bool vis[N][N];
```

matchRight：白格子现在配给谁？

```
matchRight[2][3] = {1, 3};
```

下标（2,3）是白格子的坐标；值（1,3）是它配对的黑格子坐标。一条记录已经包含两端，不是只记录白点。Right 指二分图右侧组，不是矩阵的右半边。

因为本程序坐标从 1 开始，{0,0} 不是合法格子，可用作“没有配对”的哨兵。全局对象会进行静态初始化，两个整数字段初始为 0。若改成函数内局部数组，应显式零初始化。

vis：这次寻找里是否尝试过该白点？

`vis[2][3]=true` 只表示本轮访问过它。这个点可能有配对，也可能尚无配对；访问标记不能代替配对记录。

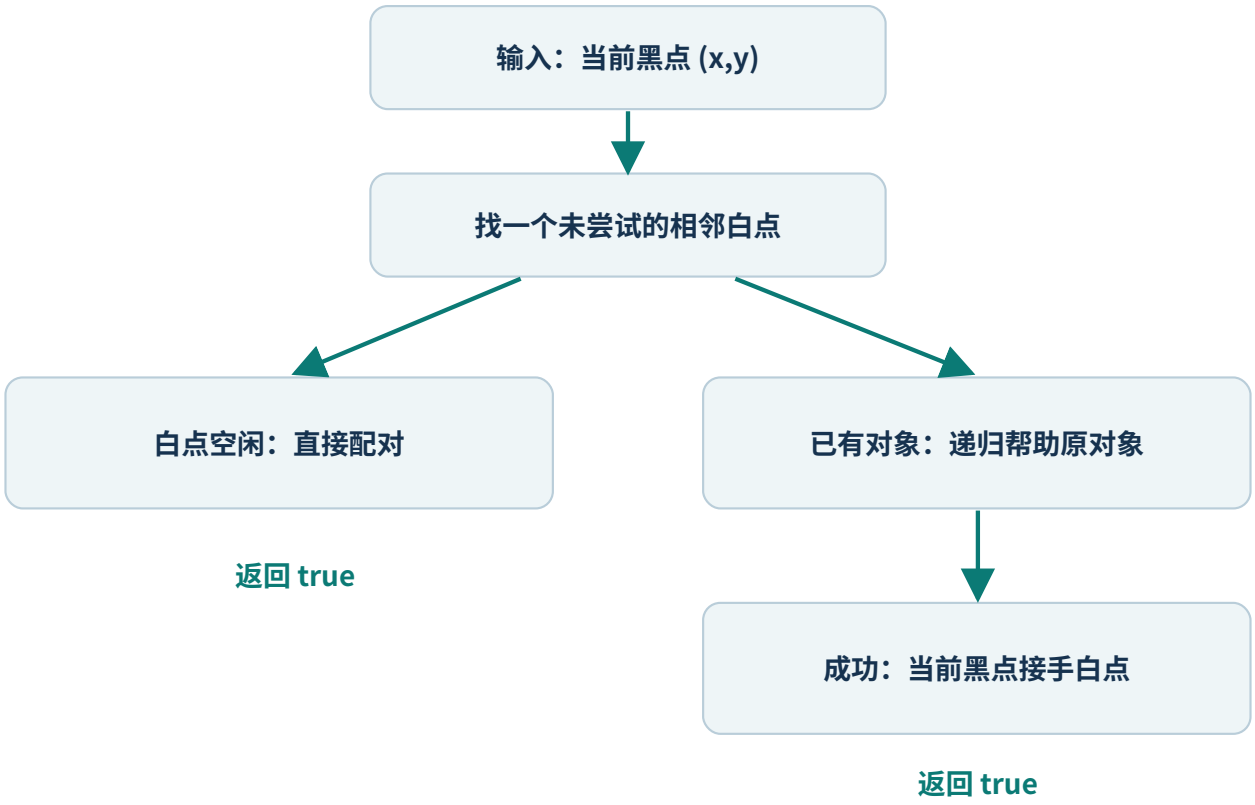
比较	matchRight	vis
内容	另一个格子的行、列坐标	true / false
改变时机	找到成功改配链时更新	某白点第一次被本轮尝试时设 true
换一个黑点启动寻找	保留已有配对	全部清空
递归进入旧黑点	继续使用同一张配对表	共享当前这一轮标记，不清空

> 虽然数组为所有坐标分配了空间，但 matchRight 和 vis 的有效读写都发生在白格子上。黑格子是由外层循环以及递归参数表示的。

学习笔记 / 16

DFS 的任务：先找空位，必要时帮助原对象改配

`dfs(x,y)` 的参数是黑格子。外层调用时，它尚未配对；递归调用时，它可能是被要求改配的旧黑格子。返回 true 表示这次寻找已经成功修改了必要的配对记录。



原对象改配失败：继续尝试其他邻居；全部失败才返回 false。

```
Point old = matchRight[nx][ny];
if (old.row == 0 || dfs(old.row, old.col)) {
    matchRight[nx][ny] = {x, y};
    return true;
}
```

真正写入配对关系的是赋值语句，不是 `vis=true`。`dfs(old.row,old.col)` 也不是只做判断：它会先实际搜索并改配，成功返回后，当前这一层再接手原白点。

> 遇到一个邻居无法改配，不要立即 return false。继续检查其余三个方向；只有整个 for 循环都失败，才说明当前黑点找不到增广链。

学习笔记 / 17

手动推演①：先给 A 配上 P

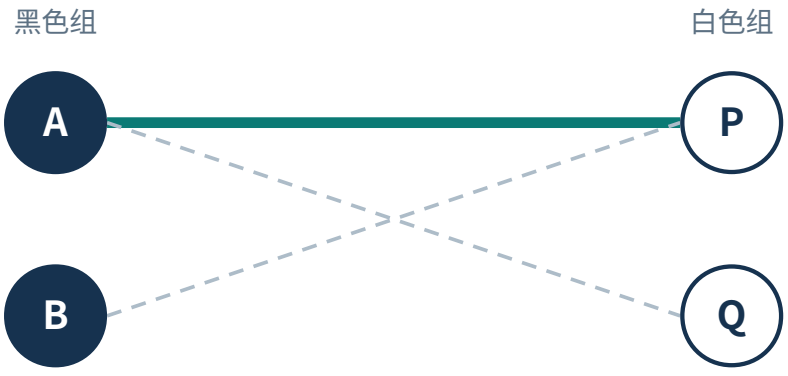
跟踪用矩阵

	1	2
1	A	Q
2	P	0
3	B	0

点	坐标	类型
A	(1,1)	黑
B	(3,1)	黑
P	(2,1)	白
Q	(1,2)	白

A 与 P、Q 相邻；B 只与 P 相邻。按代码“上、下、左、右”的顺序，从 A 开始，先遇到下方的 P。

```
调用 dfs(1,1)
 找到白点 P=(2,1)
  vis[2][1] = true
  old = matchRight[2][1] = {0,0}
  old.row == 0 成立，不执行右边的递归
  matchRight[2][1] = {1,1}
  return true
```



实线为已匹配边；虚线为可选但未匹配的边

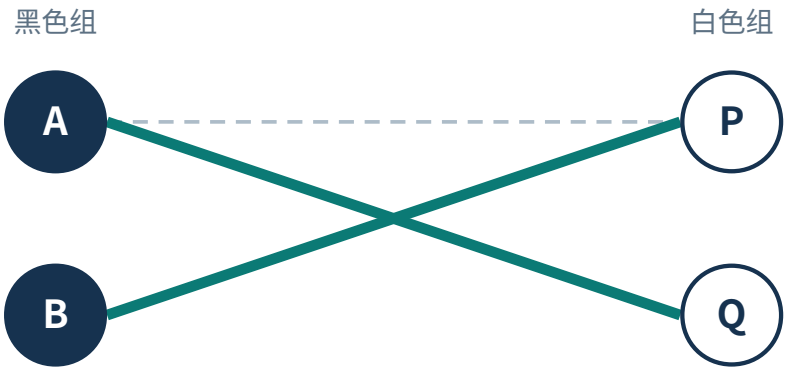
主循环收到 true，执行 matching++，当前配对数为 1。Q 仍为空闲。下一轮给 B 找配对前，会清空 vis，但保留 P 配给 A 的记录。

> 这是一个故意选取的例子：第一次配 A—P 不是最终最优安排，但算法允许后面调整，所以不需要预先猜“最聪明”的第一对。

学习笔记 / 18

手动推演②：B 想要 P，递归让 A 改配 Q

步骤	当前调用 / 动作	关键数组变化
1	外层清空 vis，调用 dfs(B)	保留 matchRight[P]=A
2	B 访问白点 P	vis[P]=true；old=A
3	P 已有对象，递归 dfs(A)	还没有把 P 抢给 B
4	A 再看到 P，但本轮已访问	跳过 P，防止绕回原处
5	A 找到空闲 Q	vis[Q]=true；matchRight[Q]=A
6	dfs(A) 返回 true	回到 dfs(B) 的 if 条件
7	B 接手 P	matchRight[P]=B
8	dfs(B) 返回 true	主循环 matching 从 1 变成 2



改配后：A—Q、B—P；总配对数由 1 变为 2

```
dfs(B)
  └ dfs(A)
       ├ P 已访问，跳过
       ├ Q 空闲，记录 Q→A
       └ 返回 true
  ├ 记录 P→B
  └ 返回 true → 主循环 matching++
```

> 里面的 return true 只返回到调用它的那一层，不会直接结束 main。每层都有自己的 x、y、nx、ny、old；递归回来后，外层仍然保留 B、P 的坐标。

图论视角：找到增广路　B—P—A—Q，沿它翻转。代码视角：递归深入找空位，再在返回过程中逐层写入新的配对。二者完全对应。

学习笔记 / 19

精读关键条件：|| 的两种成功方式

```
Point old = matchRight[nx][ny];

if (old.row == 0 || dfs(old.row, old.col)) {
    matchRight[nx][ny] = {x, y};
    return true;
}
```

左边先求值；左边成立，右边不执行

这里使用的是内置逻辑或 `||`。它从左到右求值，左边为 true 时短路，不计算右边。[4]因此 old.row 为 0 时，不会调用 dfs(0,0)。

白点状态	左条件	右边会做什么	结果
没有配对	true	不执行	直接配给当前黑点
已有配对，旧对象可改配	false	递归实际改配并返回 true	当前黑点接手该白点
已有配对，旧对象改配失败	false	返回 false	不赋值，尝试下一个邻居

拆开写，逻辑完全相同

```
if (old.row == 0) {
    matchRight[nx][ny] = {x, y};
    return true;
}

bool ok = dfs(old.row, old.col);
if (ok) {
    matchRight[nx][ny] = {x, y};
    return true;
}
// 两条路都失败：继续 for 循环，不在这里 return false。
```

> 不能把 || 改成按位或 |。后者不提供同样的短路保证，可能在 old={0,0} 时仍然递归，破坏函数的“参数必须是真实黑格子”前提。

学习笔记 / 20

vis、失败返回，以及递归中的临时记录

为什么每轮都清空 vis，递归里却不清空?

本轮的标记阻止同一个白点被重复搜索。例如 B 找 P，P 原来配给 A；递归到 A 时若又找 P，就会反复让 A 找自己。把 P 标记为已访问，才能转向其他候选。

换一个黑点开始时，情况和搜索根改变了，之前访问过的白点必须允许再次尝试。所以清空操作放在 main 的外层调用之前，而不是放在 dfs 里面。

失败会不会把原来的配对弄坏?

不会。函数只在“白点空闲”或者“递归已经成功”时写入配对，并且写入后立刻返回 true。如果某次调用最后返回 false，就说明所有分支都没成功，也没有执行成功分支里的赋值。它保留原来的匹配，只改变本轮 vis。

```
例：A–P 已匹配，B 也只邻接 P。
B → P → 递归找 A
A 看到 P 已访问，又没有别的白点 → false
B 不能抢走 P → false
原来的 A–P 保留；匹配数仍为 1。
```

调试时看到“两个白点暂时都指向 A”，是错误吗?

在上一页的成功链中，内层先写 matchRight[Q]=A，外层随后才把 matchRight[P] 从 A 改为 B。因此两次赋值之间，原始记录短暂出现 Q→A、P→A。不要把这一瞬间当成已经完成的匹配。

成功链不会在这时继续探索其他失败分支，而是立即逐层返回并更新。等最外层 dfs 完成，P→B、Q→A 才是新的有效匹配。匹配约束应在每轮外层调用结束时检查。

> 不要为了消除这个短暂状态，未经设计就提前清空 matchRight[P]；那可能让递归错误地把刚腾出的同一个白点当成空位。原写法靠共享 vis 和成功后逐层返回完成整体调整。

学习笔记 / 21

为什么整个程序一定得到最大匹配?

一轮 DFS 为什么等价于寻找增广路?

黑点枚举邻接白点，对应走一条黑→白的候选边。白点未配对，说明到达空闲终点；白点已配对，就递归到其唯一对象，对应沿匹配边白→黑走。原对象的已匹配白点已经标记过，不会马上走回原边。

标记避免重复搜索，同一轮在固定的配对关系下把所有可达候选都尝试到；若找到空闲终点，就沿调用栈返回并翻转这条链。找不到则当前根没有增广路。

每个黑点只启动一次，之前失败的点不用重试吗?

用归纳法看程序，而不是凭直觉。设已经处理了前 t 个黑点；考虑“这些黑点加全部白点”组成的子图，当前匹配是该子图的最大匹配。开始时没有黑点，空匹配当然最大。

现在加入一个新黑点 u。增加一个点，最大匹配大小至多增加 1：从任何新匹配中删去与 u 关联的一条边，就回到旧子图的匹配。

如果新图确实有更大的匹配，根据 Berge 定理一定存在增广路。它必须包含 u，否则旧子图里也有增广路，违背旧匹配最大。又因为 u 当前未匹配，它不能是增广路内部点，只能是端点。因此必须存在一条从 u 出发的增广路。

所以，从 u 做一次完整 DFS：成功就增加一对并达到新上限；失败说明新图不能增加，原匹配仍然最大。归纳继续，处理完所有黑点时得到整图最大匹配。[2]

DFS 正确增广 → 最大匹配 → 最小点覆盖 → 最大独立集

> 完整正确性分为两部分：
> ① 代码确实求出最大匹配，而不是某个随便的匹配。
> ② 本题的图是二分图，所以能用 Kőnig 定理把匹配大小换成答案。两部分缺一不可。

源码 / 第 1–52 行

完整 C++14 源码①：数据与 DFS

下列行号与附件 `grid_independent_set.cpp` 一致。第 22–23 页合起来就是完整程序；源文件不含页码和排版行号。

```
01 #include <iostream>
02 #include <cstring>
03 using namespace std;
04
05 const int N = 25;  // 题目 n,m <= 20，坐标从 1 开始
06 int n, m;
07 int a[N][N];
08
09 struct Point {
10     int row;
11     int col;
12 };
13
14 // 下标是白格子的坐标，内容是与它配对的黑格子坐标。
15 // 全局数组初始为 {0,0}，表示该白格子尚未配对。
16 Point matchRight[N][N];
17
18 // 仅记录本轮 DFS 是否尝试过某个白格子。
19 bool vis[N][N];
20
21 // 上、下、左、右；不包含斜对角。
22 int dx[4] = {-1, 1, 0, 0};
23 int dy[4] = {0, 0, -1, 1};
24
25 // 为黑格子 (x,y) 寻找配对，必要时调整已有配对。
26 bool dfs(int x, int y) {
27     for (int d = 0; d < 4; ++d) {
28         int nx = x + dx[d];
29         int ny = y + dy[d];
30
31         // 必须先检查边界，之后才能访问数组。
32         if (nx < 1 || nx > n || ny < 1 || ny > m) {
33             continue;
34         }
35         if (a[nx][ny] == 0 || vis[nx][ny]) {
36             continue;
37         }
38
39         // 当前点是黑点，相邻的可选格子一定是白点。
40         vis[nx][ny] = true;
41         Point old = matchRight[nx][ny];
42
43         // 白点空闲，或它原来的对象能够成功改配。
44         // || 短路：old.row == 0 时不会递归调用 dfs。
45         if (old.row == 0 || dfs(old.row, old.col)) {
46             matchRight[nx][ny] = {x, y};
47             return true;
48         }
49     }
50     // 所有候选都尝试失败，才返回 false。
51     return false;
52 }
```

参数始终是黑格子，候选邻居始终是白格子。第 45–48 行是“找到空位或原对象成功改配后，当前层接手”的核心。

源码 / 第 54–88 行

完整 C++14 源码②：输入、启动寻找、输出

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;
    if (n < 1 || n > 20 || m < 1 || m > 20) return 1;

    int total = 0;
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            // 按字符读取，兼容有空格的 1 0 1 和紧挨着的 101。
            char ch;
            if (!(cin >> ch) || (ch != '0' && ch != '1')) return 1;
            a[i][j] = ch - '0';
            total += a[i][j];
        }
    }

    int matching = 0;
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            // 只从值为 1 的黑格子启动一轮寻找。
            if (a[i][j] == 1 && (i + j) % 2 == 0) {
                memset(vis, 0, sizeof(vis));
                if (dfs(i, j)) {
                    ++matching;  // 一整条增广路只使配对数增加 1。
                }
            }
        }
    }

    // 最大独立集大小 = 可选点总数 - 二分图最大匹配数。
    cout << total - matching << '\n';
    return 0;
}
```

本题采用标准输入输出

题面没有要求固定文件名，因此源码不添加 freopen。读入矩阵采用 char，既兼容带空格的 `1 0 1`，也兼容紧挨着的 `101`。每个非空白字符都必须是 0 或 1。

这里是一组数据，所以全局 matchRight 初始清零即可。若改为多组数据，应在每组重新初始化配对表、计数等，不要把上一组的匹配带入下一组。

> 最终只有一个正常输出：`total - matching`。不要把调试的配对坐标、矩阵或提示词输出到标准输出中。输入缺失或非法时直接结束，不虚构答案。

学习笔记 / 24

源代码精读①：声明、输入与主循环

代码行	含义	为什么这样写
1–3	包含 iostream、cstring	分别用于输入输出、memset 清零。
5–7	N=25；n、m；二维矩阵 a	行列最大 20，1 起始坐标使用下标 1…20，容量足够。
9–12	Point 有 row、col	一个 int 不能直接保存两个坐标；结构体把它们组合起来。
14–19	全局配对表与访问表	配对跨多轮保留，访问标记只在当前轮内有效。
21–23	四个方向的行列偏移	上下左右对应行或列改变 1；绝不包含斜对角。
55–56	关闭同步、解绑 cin 与 cout	减少不必要的输入输出开销，不改变算法。
58–59	读入并检查 n、m	不让无效尺寸进入数组访问。
61–70	逐格读取字符，累加 total	total 数的是所有 1，不只统计黑点。
72–83	枚举值为 1 的黑格子	每个黑点一次外层尝试；白点由 DFS 检查。
77	清空 vis	只清本轮访问记录，不能清 matchRight。
78–79	DFS 成功，matching 加一	一次完整增广链净增加一对，不按递归层数累加。
85–87	输出 total−matching	匹配数已由 DFS 保证最大；答案公式由定理保证。

total 为什么不按颜色计数?

题目允许选择黑色 1，也允许选择白色 1。颜色只是建图分组，不能限制答案。所有可选点都属于 V，因此 total 必须把两种颜色的 1 全部算进去。

> 二维数组不是另一种算法，只是另一种存储方式。原来的一维编号 `id(x,y)` 被行列坐标替代，图的点、边和匹配规则都没有改变。

学习笔记 / 25

源代码精读②：DFS 每一步维护什么

代码行	逐句解释
26	进入 dfs(x,y)：当前要安排的黑格子是 (x,y)。递归时可能是旧对象。
27–29	依次尝试四个方向，把候选白格子的坐标放入 nx、ny。
31–34	先检查坐标是否在矩阵内。越界就 continue，不能先读 a[nx][ny]。
35–37	格子为 0，或者本轮已经访问过，就跳过。
39–40	候选白点第一次被本轮尝试，立刻标记，递归时共享这份记录。
41	用局部变量 old 保存该白点目前的黑色对象，参数不会被后续递归混淆。
45	空闲则直接成功；非空闲则递归找 old，让它尝试改配。
46	白点现在可以由当前层接手，记录其对象为 (x,y)。
47	当前这层成功完成，返回调用者；不是立刻结束整个程序。
50–51	只有四个方向全失败，才返回 false；配对保持原状。

为什么不把 vis 在返回时改回 false?

它不是在枚举所有路径时使用的“当前路径占用”标记，而是本轮可达搜索的去重记录。一个白点连向哪个旧对象是确定的；在本轮尚未找到成功链之前，配对表不变，重复走它不会带来新机会。找到成功链后则直接逐层返回。

为什么不在 main 给白点也调用 DFS?

dfs 的语义已经约定：参数是黑点，邻居是白点，matchRight 以白点坐标索引。如果把白点也当根调用，就打破了整套约定。黑点作为起点已经遍历了所有跨组边。

> 把函数读成一句话：**找一个能腾出来的相邻白点；腾不出就换下一个；成功时写记录并返回。** 不要把它误读成“只要相邻就强行配上”。

学习笔记 / 26

实际验证：样例、穷举、独立对照与边界

本手册配套源码已使用 g++ 的 C++14 模式实际编译。除了运行题目样例，还建立独立验证程序，直接调用同一份源码中的 DFS，并检查完整的配对关系。以下数字是本次实际运行结果。

验证集合	数量	对照依据
固定样例与小边界	9 个矩阵	含两组题目样例、单格、隔开格子、改配例子等。
全部 3×3 的 0/1 矩阵	512 个	独立的“选当前点 / 不选当前点”子集递推。
全部 4×4 的 0/1 矩阵	65,536 个	同上，不调用二分图匹配或 Kőnig 定理计算参考答案。
随机 1…8 行、1…8 列	500 个	独立的按行状态压缩 DP；随机种子 20261022。
20×20 规模边界	3 个	全 1、全 0、只保留一种棋盘颜色，答案为 200、0、200。
合计	66,560 个	全部与参考答案一致。

每个矩阵还检查了什么？

检查每对端点是否都为 1、颜色是否相反、是否上下左右相邻、是否重复使用黑点、配对表非空项数是否等于 matching。随后按第 11–12 页的构造恢复点覆盖及独立集，检查覆盖了每条边、选中点不相邻、大小等于 total−matching。

同一套 66,560 个案例还使用 AddressSanitizer 和 UndefinedBehaviorSanitizer 重新运行，全部通过，未报告内存访问或未定义行为错误。

```
g++ -std=c++14 -O2 -Wall -Wextra -pedantic \
    grid_independent_set.cpp -o grid_solver
```

> **验证范围的边界：** 穷举和随机测试可以发现实现错误，但不能取代一般性的数学证明。这里没有声称获得某个在线评测平台的 AC；算法正确性由前面的证明支持。

学习笔记 / 27

拓展：怎样输出真正选中的格子？

原题只需要数量，不必做本页。如果以后要求输出坐标或 X 标记，就可以利用 Kőnig 定理构造，而不是从每一对里随便选一个。

从现有配对表恢复黑点的对象

遍历白点 v，若 matchRight[v]=u，就设置 mateLeft[u]=v。这样既知道白点配给谁，也知道哪个黑点未配对。注意这是匹配结束后的额外计算，不影响原 DFS。

重新搜索交替可达集合

```
初始化 ZL、ZR 全部为 false
把所有未配对黑点加入队列，并标记 ZL

当队列不空：
  取出黑点 u
  枚举相邻白点 v
  若 u–v 是匹配边：跳过
  若 v 已在 ZR：跳过
  标记 v 属于 ZR
  令 w = matchRight[v]
  若 w 尚未在 ZL：标记 w，加入队列
```

最大匹配条件保证这个搜索不会遇到未配对白点，否则还存在增广路，应先修正匹配算法。上面黑点只沿非匹配边前进，白点沿唯一匹配边回到黑点。

最后决定选不选

可选格子类型	是否选入最优答案
黑色点	属于 ZL 就选。
白色点	不属于 ZR 就选。
值为 0 的格子	永远不选。

> 不要复用 main 最后一轮留下的 vis：它只对应一个根的临时寻找，不是“从所有未配对黑点同时出发”的全局交替搜索。恢复方案需要新的标记。

学习笔记 / 28

常见错误与可迁移的分析技巧

常见误解 / 写法	为什么错，应该怎样理解
把黑色当 0、白色当 1	颜色由坐标决定；黑白两组里的 1 全部参与。
把所有相邻边数当最大匹配数	不同边可能共用一个点。匹配要求端点不重复。
贪心见到空闲白点就永久固定	旧配对可能阻碍后面点，必须允许递归调整。
每轮清空 matchRight	等于把之前建立的配对丢掉，无法累计最大匹配。
递归内部清空 vis	会重复绕回相同白点，可能无限递归或超时。
遇到第一个失败邻居就 false	其他邻居可能能成功，必须试完整个循环。
每一次递归成功就 matching++	改配链内部只是替换对象，一整轮只净增一对。
输出 total−2×matching	每对不必两点全丢，正确减去的是最小点覆盖大小。
任意从每个配对删除一个点	可能留下非匹配边冲突，恢复方案需要交替可达性。
把斜对角也连边	改变了题意，也可能破坏二分图条件。

以后怎样想到这种方法？

看到“选择最多对象，但某些两两组合不能同时出现”，可以先建立冲突图。若冲突图能二染色，就尝试“最大独立集 = 点数 − 最大匹配”。若不能二分，不能沿用这个等式。

网格题不一定要用二维 DP。这里的“二维”只是坐标，约束发生在相邻对象之间，图论结构比矩阵形状更关键。

> 更大数据可以研究 Hopcroft–Karp 的分层增广算法；本题最多 400 个点，二维 DFS 版本已经能处理给定范围，不必先追求更复杂实现。[2]

学习笔记 / 29

自测：不用运行代码，检查是否真正理解

问题	先自己思考
① 矩阵只有一个 1，结果是多少?	它没有相邻白点，DFS 失败，这是否意味着不能选它?
② 一行 1 1 1，最大匹配数与答案各是多少?	不要把两条边同时算进匹配。
③ A 可配 P、Q；B 只能配 P，A 已配 P。	给 B 找配对时，递归的调用顺序和写入顺序是什么?
④ old={0,0} 时会执行 dfs(0,0) 吗?	区分 || 与 |。
⑤ vis[P]=true 能说明 P 已经配对吗?	访问记录和配对记录保存的信息是否相同?
⑥ 为什么匹配最大后可以只输出 total−matching?	说出中间的“最小点覆盖”。
⑦ 一个三角形图能否直接用同一公式?	先检查二分图条件。

参考答案

① 结果 1。total=1、matching=0。找不到配对不意味着不能选；孤立点可以保留。

② 最大匹配 1，对应答案 3−1=2，选两端即可。

③ 调用 dfs(B)→dfs(A)，写入先 Q→A、再 P→B。最外层成功后总匹配数只加 1。

④ 不会。左条件为 true，|| 短路，右边不执行。

⑤ 不能。它只说明本轮尝试过；是否配对、配给谁要读 matchRight。

⑥ 最大独立集与最小点覆盖互补，而二分图的最小点覆盖等于最大匹配。

⑦ 不能。三角形非二分，最大独立集为 1，点数减最大匹配却是 2。

> 复习时试着不看公式，完整说出：格子冲突如何变成边 → 黑白分组为何成立 → 配对如何调整 → 两个定理怎样把匹配数变成答案。

学习笔记 / 30

参考资料与一页复习卡

本文围绕你的具体提问重新组织解释、例子、图示和源码。下列资料用于核对定义、定理和语言语义；未复制其图片或长段原文。定理证明在正文给出，不能用测试结果替代。

[1] MIT：二分图匹配讲义

Michel X. Goemans，18.433 Combinatorial Optimization，Lecture notes on bipartite matching，2011。重点参考第 1–4 页：二分图、匹配、点覆盖、Kőnig 定理及交替搜索。

https://math.mit.edu/~goemans/18433S11/matching-notes.pdf

[2] Cornell CS 6820：Matchings

Cornell University，Fall 2024 课程站点讲义。重点参考 §1.1–1.3：匹配、增广路、无增广路等价于最大匹配，以及增广搜索算法。

https://www.cs.cornell.edu/courses/cs6820/2024fa/handouts/matchings.pdf

[3] Princeton ORF 523：Lecture 6

A. A. Ahmadi（授课），G. Hall（笔记）。第 1–2 页给出匹配与点覆盖的定义、匹配数不超过覆盖数、Kőnig 定理，以及二分图条件不可省略的说明。

https://www.princeton.edu/~aaa/Public/Teaching/ORF523/ORF523_Lec6.pdf

[4] C++ 工作草案：Logical OR operator

章节 [expr.log.or]：内置 || 的左到右求值与短路语义。配套程序另以 C++14 实际编译核验。

https://eel.is/c++draft/expr.log.or

最后用五句话串起整道题

① 只把 1 建成点，公共边变成冲突边。
② 坐标奇偶染色，每条边跨黑白组。
③ DFS 找增广路，允许旧对象改配。
④ matchRight 记配对，vis 记本轮尝试。
⑤ 最大独立集 = 全部可选点 − 最大匹配。

> 最值得记住的代码语义：**空闲就直接配；已有对象就先帮它换；换成功后当前层接手。**